Nesterov Accelerated Gradient (NAG)

Contents

1. Overview 2
2. Prerequisites 3
3. Momentum's problem: oscillations 3
4. Two pushes at once, or one after the other 5
5. The update rule 6
6. Why NAG overshoots less 7
7. The weakness: less momentum to escape a dip 9
8. NAG on real data: MNIST 9
9. Momentum and NAG in Keras 11
10. Summary 12
11. Sources 12
12. Key terms 12

Where this fits: Pipeline map step 8 (Model). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	**8 Model**	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Optimizers in deep learning; SGD with momentum.
- **Compare with:** SGD with momentum.

1. Overview

Key point: NAG is momentum with one change: it first takes the momentum jump, then measures the gradient at the point where that jump lands (the **look-ahead point** (G-1127)), and corrects from there. Seeing the slope ahead lets it brake before the minimum, so it overshoots less than momentum.

Nesterov accelerated gradient (G-1315; NAG; Nesterov 1983, adapted to deep learning by Sutskever et al. 2013) is a small upgrade of momentum (G-1258: gradient descent that keeps a running average of past steps, its velocity, and moves by it). Momentum's speed is its strength, but near the minimum the same speed makes it overshoot and swing (**overshooting**, G-1430). NAG damps those swings.

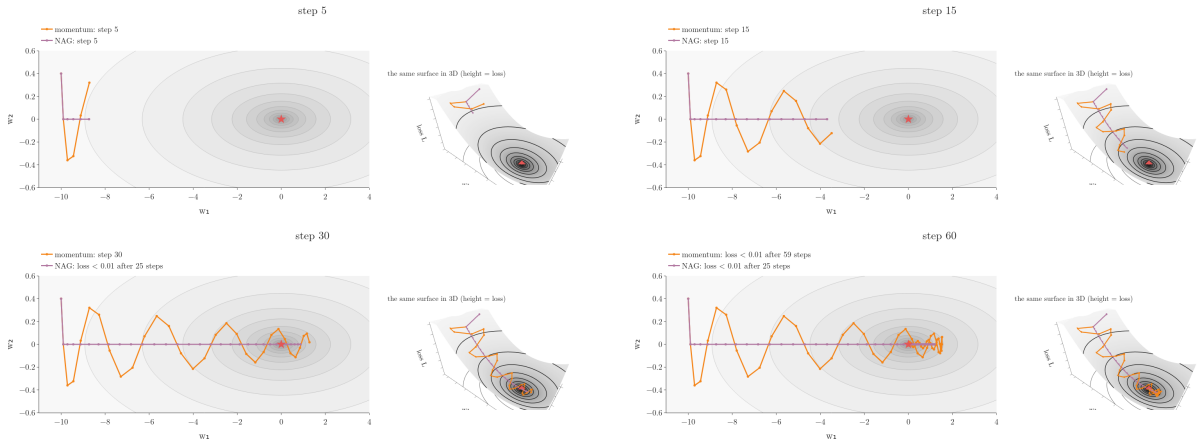


Figure 1: Momentum and NAG along the same narrow valley, both with learning rate 0.01 and $\beta = 0.9$. Momentum swings across the valley and past the minimum; NAG's swings are much smaller

Figure 1 is a **contour map** (the surface seen from above; each line joins points of equal loss; how to read a contour map), with the same surface in 3D at its right (height = the loss; the lines on it are the lines of the map). The surface is the narrow valley of one learning rate for every direction (its Figure 4 tilts it from the side view to this top view). The loss of the two weights w_1 and w_2 is

$$L(w_1, w_2) = \tfrac{1}{2}\left(w_1^2 + 100\, w_2^2\right)$$

and at the starting point $(-10, 0.4)$:

$$L(-10, 0.4) = \tfrac{1}{2}(100 + 16)$$

$$= 58$$

A long, narrow valley with steep sides and a gently sloping floor. On the map, lines close together mean steep (across the valley), lines far apart mean flat (along it), and the star is the lowest point.

Figure 1 shows the difference. On this valley momentum needs 59 steps to bring the loss below 0.01; NAG needs 25.

2. Prerequisites

- Momentum's update rule: the **velocity** (G-2085), and **overshooting** (G-1430).
- The exponentially weighted moving average: the **decay factor** (G-553) β, the share of the old average kept at each step.

3. Momentum's problem: oscillations

> **Key point:** Momentum reaches the neighbourhood of the minimum fast, then wastes many steps swinging past it. A smaller β reduces the swings, but also the speed.

Momentum covers the early part of the road quickly: in the valley of Figure 1 it is close to the minimum within about 20 steps. Then it crosses the minimum, comes back, crosses again, and only settles once its **velocity** (G-2085) has died away.

Figure 2 shows the swings on the simplest loss, the bowl

$$L(w) = \tfrac{1}{2}w^2$$

from $w = -10$ with **learning rate** (G-1068; the size of each step) 0.1.

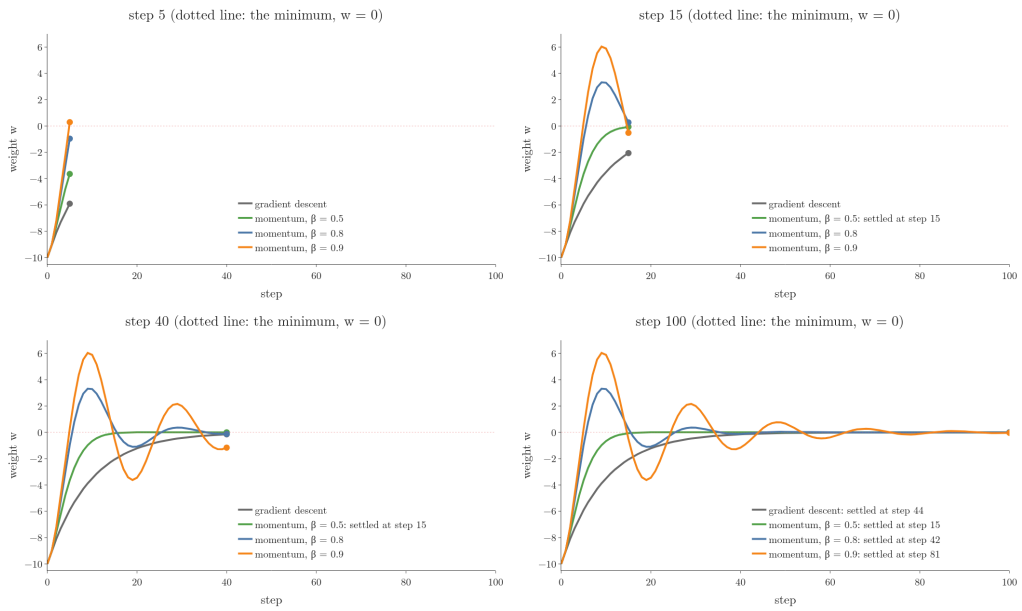


Figure 2: The weight on the bowl $L = w^2/2$ step by step: plain gradient descent and momentum with $\beta = 0.5$, 0.8 and 0.9. The legend shows the step from which each run stays within 0.1 of the minimum

	First within 1 of the minimum	Furthest past the minimum	Stays within 0.1 from step
Gradient descent	step 22	0	44
Momentum, $\beta = 0.5$	step 9	0	15
Momentum, $\beta = 0.8$	step 5	3.32	42
Momentum, $\beta = 0.9$	step 5	6.04	81

- **Getting near is fast.** With $\beta = 0.9$ or 0.8, momentum is within 1 of the minimum after 5 steps; gradient descent needs 22.
- **Settling is slow.** With $\beta = 0.9$ the weight shoots 6.04 past the minimum and keeps swinging until step 81, later than plain gradient descent (step 44).
- **A smaller β swings less.** With $\beta = 0.5$ there is no overshoot at all and the run settles by step 15.

Lowering the **decay factor** (G-553) β tames the swings, at the cost of some speed at the start (9 steps instead of 5 to get near). On the more complex, non-convex losses of neural networks (**non-convex functions**, G-1333) the **oscillations** (G-1410) cost even more time. NAG offers another fix: keep the speed, and reduce the oscillations.

4. Two pushes at once, or one after the other

> **Key point:** Momentum adds the momentum jump and the gradient step, both computed at the current point. NAG first makes the momentum jump, then computes the gradient where it lands.

Write momentum's update in one line by putting v_t into $w_{t+1} = w_t - v_t$:

$$w_{t+1} = w_t - \beta\, v_{t-1} - \eta\, \nabla L(w_t)$$

Each step has two parts: the push of the past **velocity** (G-2085), βv_{t-1}, and the push of the current gradient, $\eta\, \nabla L(w_t)$. Plain gradient descent has only the second. Momentum computes both at the current point w_t and applies them together.

NAG changes only **where the gradient is computed**. NAG first applies the momentum part alone, which takes it to a **look-ahead point** (G-1127). It computes the gradient there and then moves on, or back, according to that gradient (Goodfellow et al. 2016, §8.3.3).

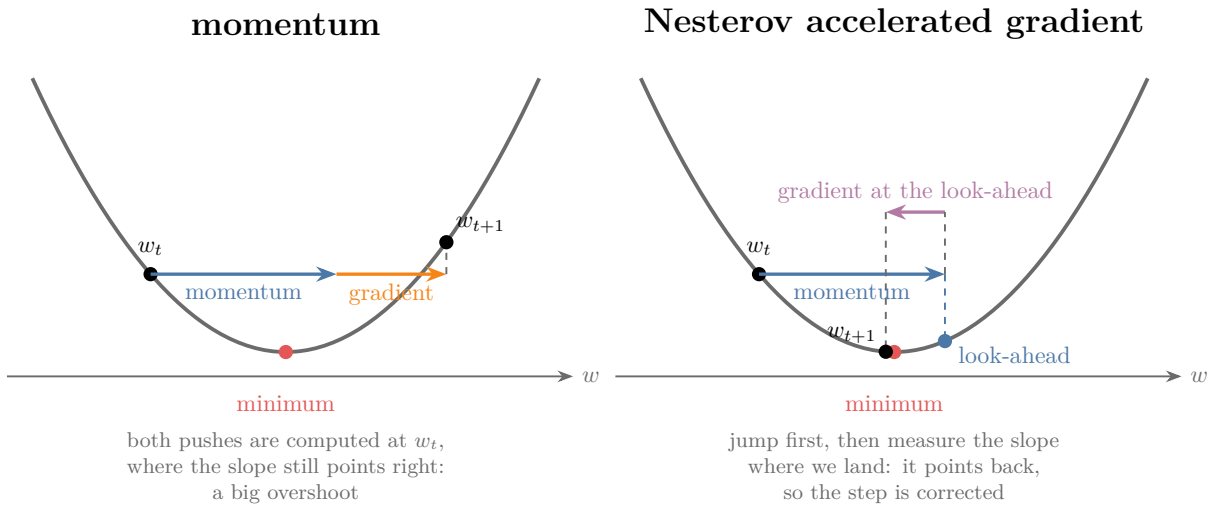


Figure 3: One step near the minimum. Left, momentum: the momentum jump and the gradient step are both computed at w_t, where the slope still points right, so the step overshoots. Right, NAG: jump first, then measure the slope at the look-ahead point, which points back, so the step is corrected

Figure 3 shows one step near the minimum. Hinton's lecture slides put the difference in one line: first make a big jump in the direction of the previous accumulated gradient, then measure the gradient where you end up and make a correction; it is better to correct a mistake after you have made it (Hinton 2012, lecture 6c).

5. The update rule

> **Key point:** NAG uses momentum's two lines unchanged, and adds one line before them: the **look-ahead point** (G-1127). Only the point where the gradient is taken changes.

1. **In words:** find where the momentum jump alone would take us. Take the gradient there. Build the **velocity** (G-2085) from the old velocity and that gradient, and move from the current point.

2. **Formula:**

$$w_{\text{la}} = w_t - \beta\, v_{t-1}$$

$$v_t = \beta\, v_{t-1} + \eta\, \nabla L(w_{\text{la}})$$

$$w_{t+1} = w_t - v_t$$

3. **Example:** the same start as for momentum: the bowl of Figure 2 (its gradient at w is w), $w_0 = -10$, $\eta = 0.1$, $\beta = 0.9$, $v_0 = 0$.

   Step 1, the same as momentum:

$$w_{\text{la}} = -10 - 0.9 \times 0 = -10$$

$$v_1 = 0.1 \times (-10) = -1$$

$$w_1 = -10 - (-1) = -9$$

   Step 2: the look-ahead point is

$$w_{\text{la}} = -9 - 0.9 \times (-1)$$

$$= -8.1$$

   Its gradient is -8.1, not -9, so

$$v_2 = 0.9 \times (-1) + 0.1 \times (-8.1)$$

$$= -1.71$$

$$w_2 = -9 - (-1.71)$$

$$= -7.29$$

   Momentum reached -7.2. NAG's step is a little shorter, because the slope at the look-ahead point is gentler (Notebook).

Putting v_t into the last line gives the whole step:

$$w_{t+1} = w_t - \beta v_{t-1}$$

$$-\eta \nabla L(w_{\text{la}})$$

It is the distance to the look-ahead point plus a gradient step taken from the look-ahead point.

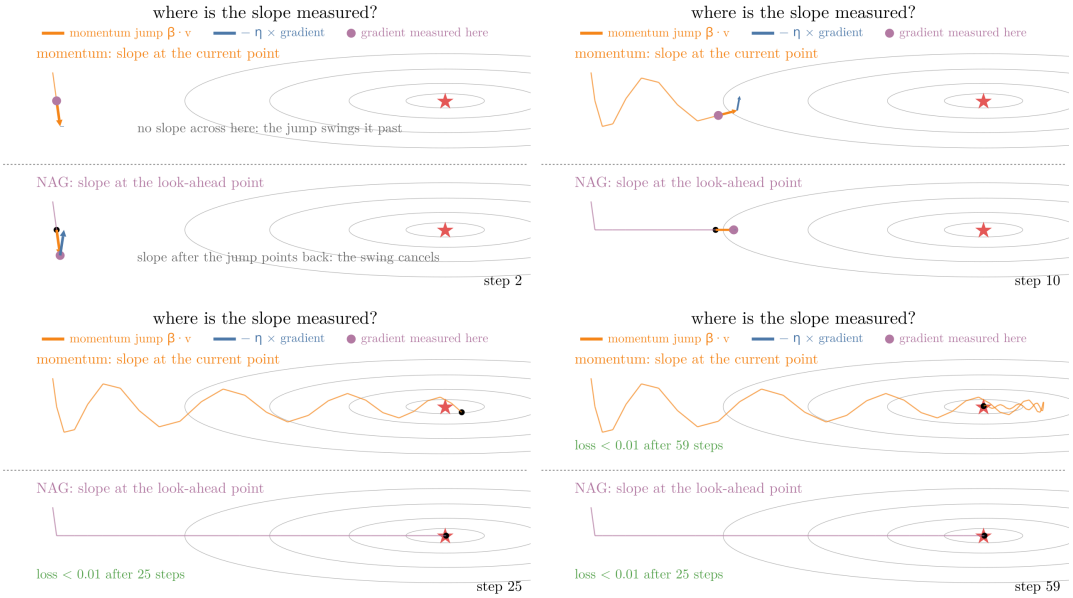


Figure 4: Momentum (top) and NAG (bottom) step by step on the valley of Figure 1. Orange: the jump βv_{t-1}; blue: $-\eta$ times the gradient; purple dot: where the gradient is measured. Momentum measures at the current point, NAG at the look-ahead point

Figure 4 applies the rule on the valley of Figure 1. Watch step 2: momentum measures the slope at a point on the valley floor, where nothing pushes back, so the jump carries it across; NAG measures at the look-ahead point on the far wall, whose slope points back and cancels the swing.

6. Why NAG overshoots less

> **Key point:** When momentum is about to carry the weights past the minimum, the slope at the **look-ahead point** (G-1127) already points back. NAG feels that and brakes one step earlier.

Follow a ball rolling down towards the minimum on a curve.

Momentum. At every point it computes the momentum and the slope at that same point. Far from the minimum, both push forward and the ball speeds up. Past the minimum the slope turns and pushes back, but the built-up momentum dominates, so the ball keeps going: it overshoots, makes a U-turn, comes back, makes a smaller U-turn, and so on until it stops.

NAG. Near the minimum, the momentum jump alone already lands past the minimum. The slope there points back, so the gradient step pulls the ball back towards the minimum, and it stops much nearer to it. NAG makes smaller U-turns and settles sooner. NAG has a sense of where it is going, so it slows down before the hill slopes up again (Ruder 2016, §4.2).

If the momentum jump is a poor move that increases the loss, the gradient at the look-ahead point points back towards w_t more strongly than the gradient at w_t itself, which gives a larger and more timely correction to the **velocity** (G-2085). On an oblong bowl, momentum swings strongly across the steep direction while NAG avoids these swings almost entirely (Sutskever et al. 2013, §2.1).

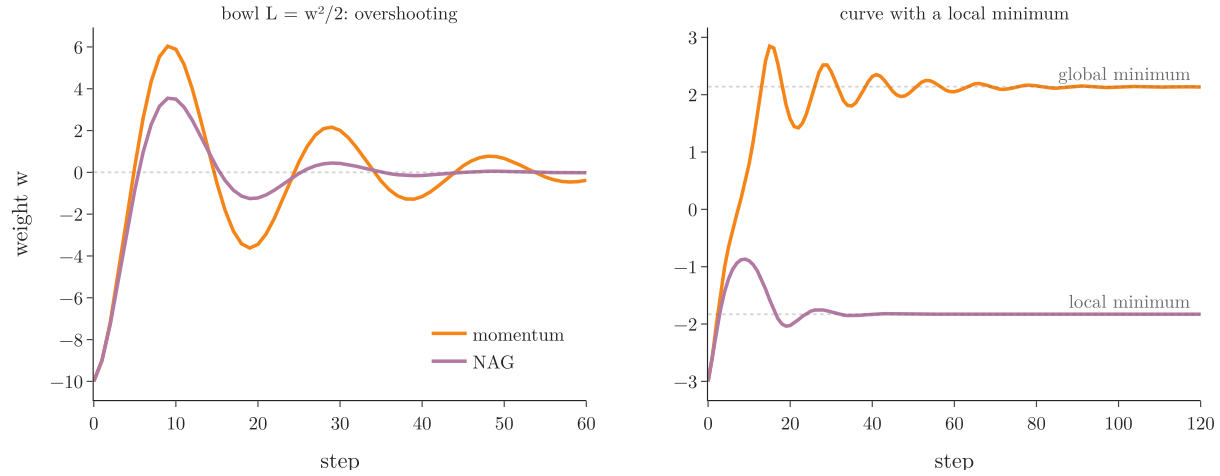


Figure 5: The weight over the steps. Left: the bowl $L = w^2/2$ from $w = -10$, $\eta = 0.1$, $\beta = 0.9$; NAG (purple) swings less far past the minimum and settles sooner than momentum (orange). Right: a curve with a local minimum from $w = -3$, $\eta = 0.05$, $\beta = 0.9$; momentum rolls on to the global minimum, NAG stays in the local one

Figure 5 (left) and Figure 1 measure the effect (Notebook):

	Momentum	NAG
Bowl: furthest past the minimum	6.04	3.55
Bowl: stays within 0.1 of the minimum from step	81	42
Valley: steps until the loss is below 0.01	59	25

Extra: On convex problems (**convex functions**, G-476) with exact gradients, Nesterov's method provably converges faster than plain gradient descent: the excess loss falls like $1/k^2$ after k steps instead of $1/k$ (Nesterov 1983; Goodfellow et al. 2016, §8.3.3). With noisy mini-batch gradients that guarantee is lost (Goodfellow et al. 2016, §8.3.3); the benefit that remains in practice is the damping seen here.

7. The weakness: less momentum to escape a dip

> **Key point:** The damping that removes **overshooting** (G-1430) also removes some of the extra push. Where momentum rolls out of a small local minimum, NAG can stay stuck in it.

Damping the **oscillations** (G-1410) has a possible downside. On a loss with a small dip on the way to a deeper one, momentum may build up enough speed to roll over the bump and out of the dip; NAG brakes earlier, so it may not gain that speed and can settle in the **local minimum** (G-1110).

Figure 5 (right) shows exactly this on the curve with a small dip:

$$L(w) = \frac{(w^2 - 4)^2}{8} - 0.6w$$

With the same settings, momentum ends in the **global minimum** (G-848) near $w = 2.14$ and NAG in the local one near -1.83. Over 12 settings (two starts, three learning rates, two values of β), momentum reached the global minimum in 8 and NAG in 4, and NAG never escaped where momentum did not (Notebook). On such losses another optimizer may do better.

8. NAG on real data: MNIST

> **Key point:** On MNIST with $\beta = 0.9$, momentum and NAG train almost identically. With $\beta = 0.99$, where momentum's swings matter more, NAG's training is steadier and ends slightly lower.

We repeat the momentum experiment on **MNIST** (G-1249; images of handwritten digits):

- **data:** 10,000 training images, each with 784 pixel **features** (G-772; input variables) and the digit as **target** (G-1949; the output we predict), and the 10,000 test images for validation (checking accuracy on images the network did not train on);
- **network:** hidden layers of 128 and 64 **ReLU** (G-1668; each node outputs its input if positive, else 0) nodes and a softmax output (10 probabilities, one per digit);
- **training:** learning rate 0.01, **batch size** (G-267) 64, 20 **epochs** (G-696), 3 seeds.

Only `nesterov` changes, at $\beta = 0.9$ and at $\beta = 0.99$.

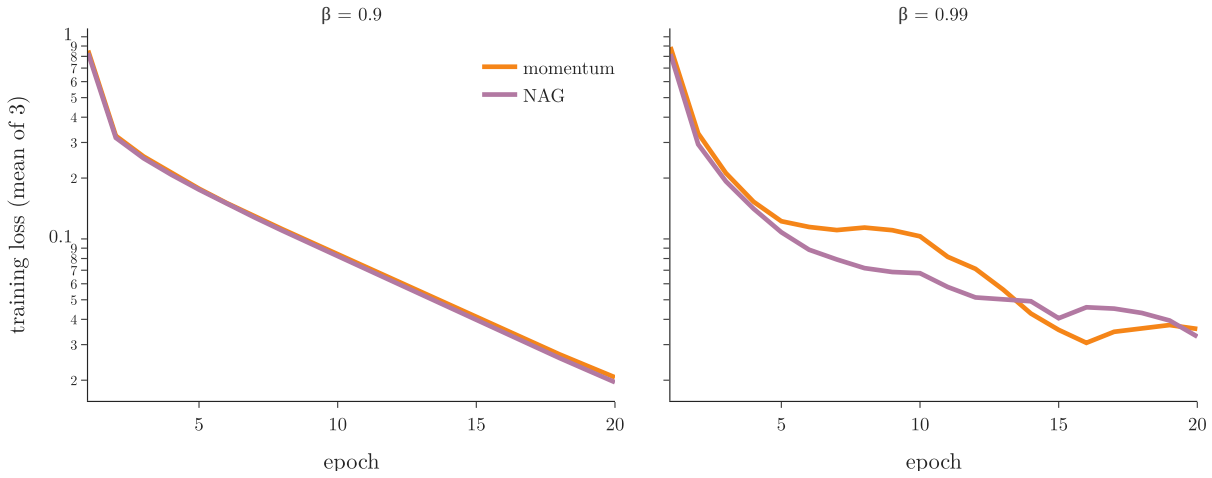


Figure 6: Training loss on MNIST per epoch, mean of 3 seeds. Left: $\beta = 0.9$, the two curves overlap. Right: $\beta = 0.99$, NAG (purple) mostly below momentum (orange)

In Figure 6 each curve is the training loss after each epoch. A **log scale** axis: each labelled line (such as 0.1 and 1) is 10 times the one below it, the small numbers between mark 2, 3, ... times that line, so equal vertical distances mean equal ratios, and a straight line down is a steady percentage drop per step. A lower curve means a better fit.

Figure 6 and the Notebook give, as means over the 3 seeds:

	$\beta = 0.9$: momentum	$\beta = 0.9$: NAG	$\beta = 0.99$: momentum	$\beta = 0.99$: NAG
Training loss, epoch 5	0.177	0.175	0.122	0.108
Training loss, epoch 20	0.021	0.020	0.036	0.033
Validation accuracy, epoch 20	0.950	0.949	0.944	0.946
Epochs where the loss went up	0	0	4	1

With $\beta = 0.9$ the two are practically the same: the gradient at the **look-ahead point** (G-1127) is almost the gradient at the current point. With $\beta = 0.99$ momentum's **velocity** (G-2085) is ten times longer-lived, its training loss went up from one epoch to the next in 4 epochs, against 1 for NAG, and NAG ends with a slightly lower loss and slightly higher validation accuracy. The differences are small and vary from seed to seed: in one of the three seeds momentum ended lower (Notebook).

Sutskever et al. (2013, §2.1) found the same pattern: NAG changes the velocity in a quicker

and more responsive way, which lets it behave more stably than momentum in many situations, especially for higher values of β.

9. Momentum and NAG in Keras

Key point: Keras' `SGD` class covers all three: `momentum=0` is plain SGD, `momentum=0.9` is momentum, and `momentum=0.9, nesterov=True` is NAG.

Python: Three optimizers from one class.

```
import keras
sgd = keras.optimizers.SGD(learning_rate=0.01, momentum=0.0)
mom = keras.optimizers.SGD(learning_rate=0.01, momentum=0.9)
nag = keras.optimizers.SGD(learning_rate=0.01, momentum=0.9,
                           nesterov=True)
```

`momentum` is the **decay factor** (G-553) β, usually 0.9 or 0.5; `nesterov` switches the look-ahead on.

Extra: Keras never computes the gradient at a separate **look-ahead point** (G-1127). Its weights are the look-ahead point itself: write $u_t = w_t - \beta v_{t-1}$, so $w_t = u_t + \beta v_{t-1}$. One step per line:

$$u_{t+1} = w_{t+1} - \beta v_t$$

Since $w_{t+1} = w_t - v_t$:

$$= (w_t - v_t) - \beta v_t$$

Replace w_t by $u_t + \beta v_{t-1}$:

$$= u_t + \beta v_{t-1} - v_t - \beta v_t$$

Put in the **velocity** (G-2085), whose gradient is taken at the look-ahead point u_t:

$$v_t = \beta v_{t-1} + \eta \nabla L(u_t)$$

The two βv_{t-1} cancel:

$$u_{t+1} = u_t - \beta\, v_t - \eta \nabla L(u_t)$$

This is a step that needs only the gradient g at the stored weights. With $m = -v$ this is Keras' rule (Keras `SGD` documentation):

$$m \leftarrow \beta m - \eta g$$

$$w \leftarrow w + \beta m - \eta g$$

The Notebook checks it: Keras' weights $-10, -8.1, -5.75, -3.27, \ldots$ are exactly our look-ahead points.

10. Summary

	Momentum	NAG
Gradient taken at	the current point w_t	the look-ahead point $w_t - \beta v_{t-1}$
Near the minimum	overshoots, large swings	brakes earlier, small swings
Bowl $L = w^2/2$: furthest past the minimum	6.04	3.55
Valley: steps to loss < 0.01	59	25
Small local dip	more push to roll out	can stay stuck
Keras	`SGD(momentum=0.9)`	`SGD(momentum=0.9, nesterov=True)`

- NAG is momentum with the gradient measured after the momentum jump, at the **look-ahead point** (G-1127).
- Seeing the slope ahead lets NAG correct the **velocity** (G-2085) before it overshoots, so it oscillates less and often converges faster.
- The same damping can leave it in a small **local minimum** (G-1110) that momentum would roll out of.
- On MNIST it matches momentum at $\beta = 0.9$ and is slightly steadier at $\beta = 0.99$.

11. Sources

Built from

- CampusX, "Nesterov Accelerated Gradient (NAG) Explained in Detail | Animations | Optimizers in Deep Learning", YouTube, https://www.youtube.com/watch?v=rKG9E6rce1c

Other references

- Goodfellow, I., Bengio, Y. and Courville, A. (2016). *Deep Learning*. MIT Press. §8.3.3 Nesterov momentum.
- Hinton, G. (2012). *Neural Networks for Machine Learning*, Coursera, lecture 6c: The momentum method (slides).
- Nesterov, Y. (1983). A method for solving the convex programming problem with convergence rate $O(1/k^2)$. *Soviet Mathematics Doklady* 27, 372–376.
- Ruder, S. (2016). An overview of gradient descent optimization algorithms. arXiv:1609.04747. §4.2 Nesterov accelerated gradient.
- Sutskever, I., Martens, J., Dahl, G. and Hinton, G. (2013). On the importance of initialization and momentum in deep learning. ICML 2013. §2.1.
- Keras API documentation: `SGD` optimizer (`momentum`, `nesterov`), keras.io/api/optimizers/sgd.

12. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Look-ahead point (G-1127)	The point that the momentum step alone would take the weights to, $w_t - \beta v_{t-1}$; Nesterov momentum measures the gradient there instead of at the current weights, so it corrects sooner and overshoots less.
Nesterov accelerated gradient (NAG) (G-1315)	A version of momentum that first takes the momentum jump and then measures the gradient where it lands (the look-ahead point); seeing the slope ahead lets it brake before the minimum and swing less.
Oscillation (G-1410)	Swinging back and forth past the minimum before settling.
Damping (G-528)	Making an optimiser's back-and-forth swings (oscillations) smaller: NAG brakes before it overshoots, so its path settles faster than momentum's, though it can then stay in a small dip.
Momentum (optimizer) (G-1258)	Gradient descent that keeps part of its past movement: each step follows an average of past gradients in which older ones count less and less (a velocity, an exponentially decaying average).
Overshooting (G-1430)	Moving past the minimum because of the built-up velocity, then swinging back.
Velocity v (G-2085)	The direction and size of the current move, built from past gradients: $v_t = \beta v_{t-1} + \eta \nabla L(w_t)$.
Decay factor β (G-553)	In momentum, the share of the previous update (the velocity) kept at each step; 0 gives plain gradient descent, usually 0.9.
Learning rate (η) (G-1068)	How strongly each update changes the model; in gradient descent, the number the slope is multiplied by to get the step size.
Non-convex function (G-1333)	A function whose curve is not one simple bowl: some straight line between two of its points (a chord) lies below part of the curve. It can have several local minima.
Convex function (G-476)	A function where a straight line between any two points of its curve never goes below the curve; it has a single minimum.
Local minimum (G-1110)	A point lower than everything around it, but not the lowest overall.
Global minimum (G-848)	The lowest point of the whole function.

Term	Meaning
MNIST (G-1249)	A dataset of about 70,000 handwritten-digit images of 28×28 pixels.
Feature (G-772)	One piece of information about each example that a model uses (e.g. a student's CGPA).
Target (G-1949)	The output column we predict, such as the class; also called the label.
ReLU (G-1668)	Rectified linear unit, the activation $\max(0, z)$: it passes a positive input unchanged and turns a negative one into 0; the usual default activation for hidden layers.
Batch size (G-267)	The number of rows in each batch; a hyperparameter.
Epoch (G-696)	One full update of the parameters using the whole training set.